

Course « *Deep Learning* »

Sid-Ahmed Berrani

ENSIA

2024 – 2025

Convolutional Neural Network (CNN)

Reference: In 1959, Hubel & Wiesel performed a set of experiments in order to understand how the visual cortex process visual information in the brain:

- Neural activities of the visual striate of a cat have been recorded.
- Certain neurons are triggered when light stimuli has a specific angle and position.

In 1989, Yann Le Cun proposed a CNN model that has been trained using back-propagation.

CNN becomes very popular when it outperformed the other models at the ImageNet Challenge.

Popular CNN models that wins the ImageNet challenge: **AlexNet** (2012), **ZFNet** (2013), **GoogLeNet** & VGG (2014), **ResNet** (2015)

CNN is a model that is composed of neurons in the form of filters applied following layers called convolutional layers.

CNN has replaced the feed-forward NN for image processing application: Classification, object detection, segmentation...

Image Processing

What we see



What a machine sees

0	2	15	0	0	11	10	0	0	0	0	9	9	0	0	0
0	0	0	4	60	157	236	255	255	177	95	61	32	0	0	29
0	10	16	119	238	255	244	245	243	250	249	255	222	103	10	0
0	14	170	255	255	244	254	255	253	245	255	249	253	251	124	1
2	98	255	228	255	251	254	211	141	116	122	215	251	238	255	49
13	217	243	255	155	33	226	52	2	0	10	13	232	255	255	36
16	229	252	254	49	12	0	0	7	7	0	70	237	252	235	62
6	141	245	255	212	25	11	9	3	0	115	236	243	255	137	0
0	87	252	250	248	215	60	0	1	121	252	255	248	144	6	0
0	13	113	255	255	245	255	182	181	248	252	242	208	36	0	19
1	0	5	117	251	255	241	255	247	255	241	162	17	0	7	0
0	0	0	4	58	251	255	246	254	253	255	120	11	0	1	0
0	0	4	97	255	255	255	248	252	255	244	255	182	10	0	4
0	22	206	252	246	251	241	100	24	113	255	245	255	194	9	0
0	111	255	242	255	158	24	0	0	6	39	255	232	230	56	0
0	218	251	250	137	7	11	0	0	0	2	62	255	250	125	3
0	173	255	255	101	9	20	0	13	3	13	182	251	245	61	0
0	107	251	241	255	230	98	55	19	118	217	248	253	255	52	4
0	18	146	250	255	247	255	255	255	249	255	240	255	129	0	5
0	0	23	113	215	255	250	248	255	255	248	248	118	14	12	0
0	0	6	1	0	52	153	233	255	252	147	37	0	0	4	1
0	0	5	5	0	0	0	0	0	14	1	0	6	6	0	0

Image Processing & Computer Vision

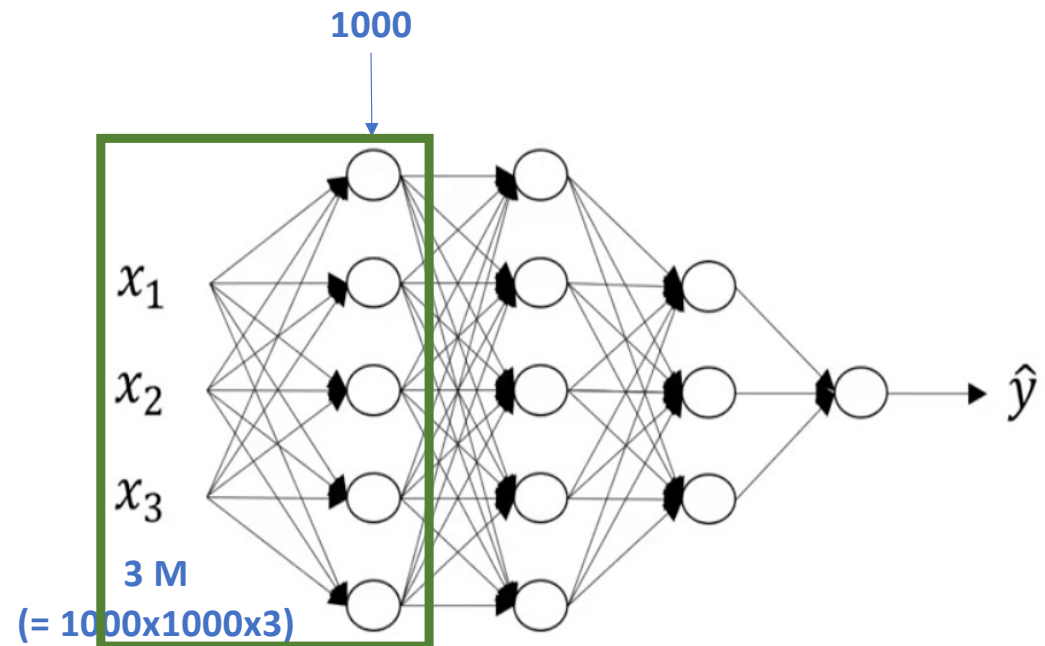
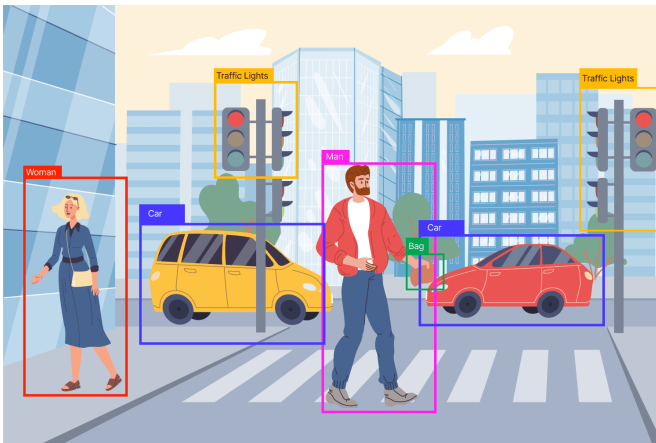
Image classification



1000x1000

→ Cat? (0/1)

Object detection



The number of
parameters in the 1st
layer = 1 billion + 1000

Image Processing & Computer Vision



Vertical
edges



Horizontal
edges

Image Processing & Computer Vision

Vertical edge detection

3	0	1	2	7	4
1	5	8	9	3	1
2	7	2	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

Image
6x6

Convolution
*

1	0	-1
1	0	-1
1	0	-1

Filter
3x3

=

Image Processing & Computer Vision

Vertical edge detection

3	0	1	2	7	4
1	5	8	9	3	1
2	7	2	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

Image
6x6

Convolution
*

1	0	-1
1	0	-1
1	0	-1

Filter
3x3

=

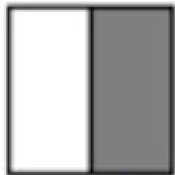
Output image
4x4

Image Processing & Computer Vision

Vertical edge detection

10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0

Image
6x6



Convolution
*

1	0	-1
1	0	-1
1	0	-1

Filter
3x3



=

0	30	30	0
0	30	30	0
0	30	30	0
0	30	30	0

Output image
4x4

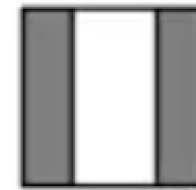


Image Processing & Computer Vision

Edge detection

1	0	-1
1	0	-1
1	0	-1

Vertical

1	1	1
0	0	0
-1	-1	-1

Horizontal

1	0	-1
2	0	-2
1	0	-1

Sobel filter

3	0	-3
10	0	-10
3	0	-3

Schaar filter

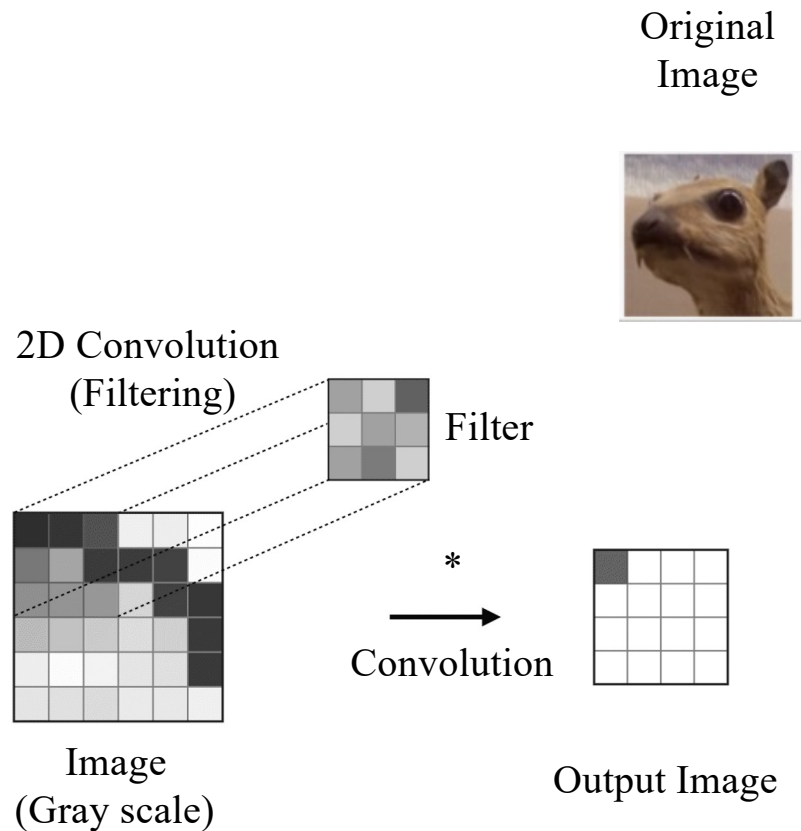
3	0	1	2	7	4
1	5	8	9	3	1
2	7	2	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

*

w_1	w_2	w_3
w_4	w_5	w_6
w_7	w_8	w_9

=

Examples of other filters used in image processing



Original Image



Filtering (Convolution)



Impulse (Neutral)

Edge detection

Sharpening

Averaging filter

Gaussian filter

Filter	Convolved Image
$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$	
$\begin{bmatrix} 1 & 0 & -1 \\ 0 & 0 & 0 \\ -1 & 0 & 1 \end{bmatrix}$	
$\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$	
$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$	
$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$	
$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$	
$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$	

Image Processing & Computer Vision

Convolution & Padding

10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0

Image
 $n \times n$

Convolution
 $*$

1	0	-1
1	0	-1
1	0	-1

Filter
 $f \times f$

=

0	30	30	0
0	30	30	0
0	30	30	0
0	30	30	0

Output image
 $(n-f+1) \times (n-f+1)$

Image Processing & Computer Vision

Convolution & Padding

0	0						
0	10	10	10	0	0	0	
	10	10	10	0	0	0	
	10	10	10	0	0	0	
	10	10	10	0	0	0	
	10	10	10	0	0	0	
	10	10	10	0	0	0	

Image

$n \times n$

Padding $p = 1$

Convolution

*

1	0	-1
1	0	-1
1	0	-1

Filter

$f \times f$

=

0	30	30	0
0	30	30	0
0	30	30	0
0	30	30	0

Output image
 $(n-f+1) \times (n-f+1)$

Output image

$n \times n$

In the general case : Output image
 $(n+2p-f+1) \times (n+2p-f+1)$

Image Processing & Computer Vision

Strided convolution

2	3	7	4	6	2	9
6	6	9	8	7	4	3
3	4	8	3	8	9	7
7	8	3	6	6	3	4
4	2	1	8	3	4	6
3	2	4	1	9	8	3
0	1	3	9	2	1	4

Image
7 x 7

*

3	4	4
1	0	2
-1	0	3

Filter
3 x 3

=

Convolution with a stride 2

Image Processing & Computer Vision

Strided convolution

2	3	7	4	6	2	9
6	6	9	8	7	4	3
3	4	8	3	8	9	7
7	8	3	6	6	3	4
4	2	1	8	3	4	6
3	2	4	1	9	8	3
0	1	3	9	2	1	4

Image
 $n \times n$

*

3	4	4
1	0	2
-1	0	3

Filter
 $f \times f$

=

Output
 $r \times r$

Convolution with:

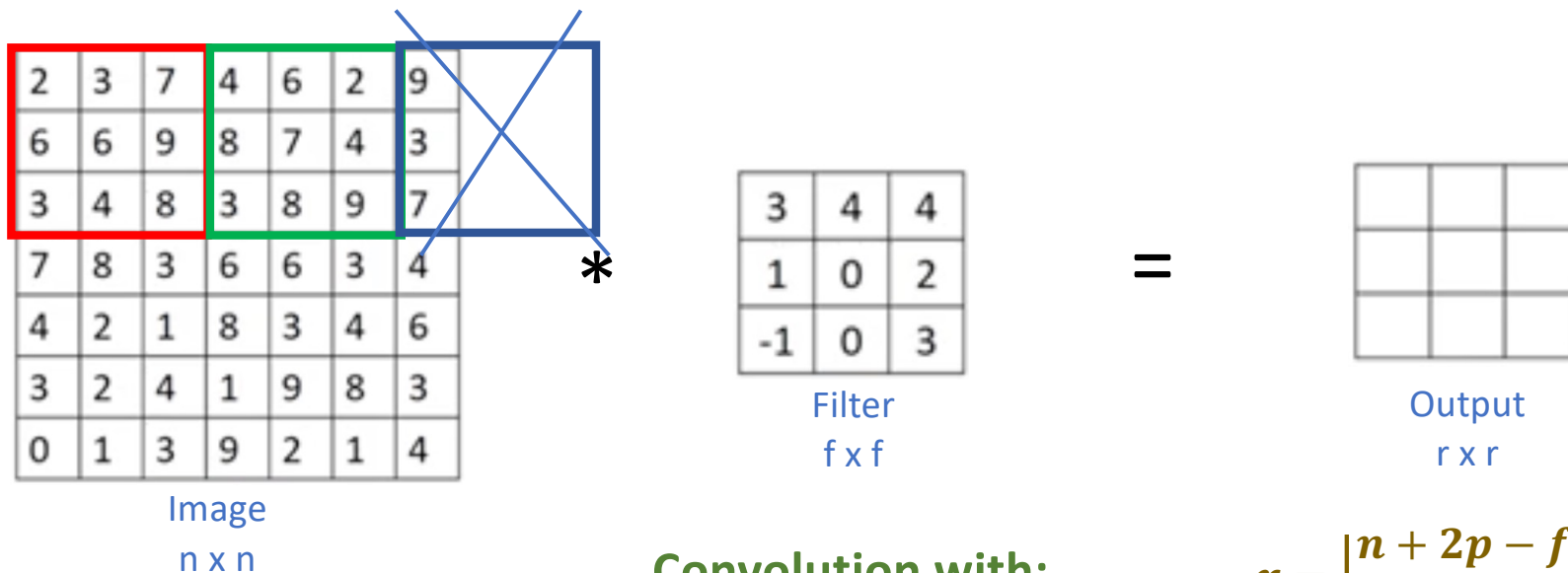
- A padding p
- A stride s

$$r = \left\lfloor \frac{n + 2p - f}{s} + 1 \right\rfloor$$

Example: $n = 7$; $f = 3$; $p = 0$; $s = 2$ $\Rightarrow$ $r = 3$

Image Processing & Computer Vision

Strided convolution



Convolution with:

- A padding p
- A stride s

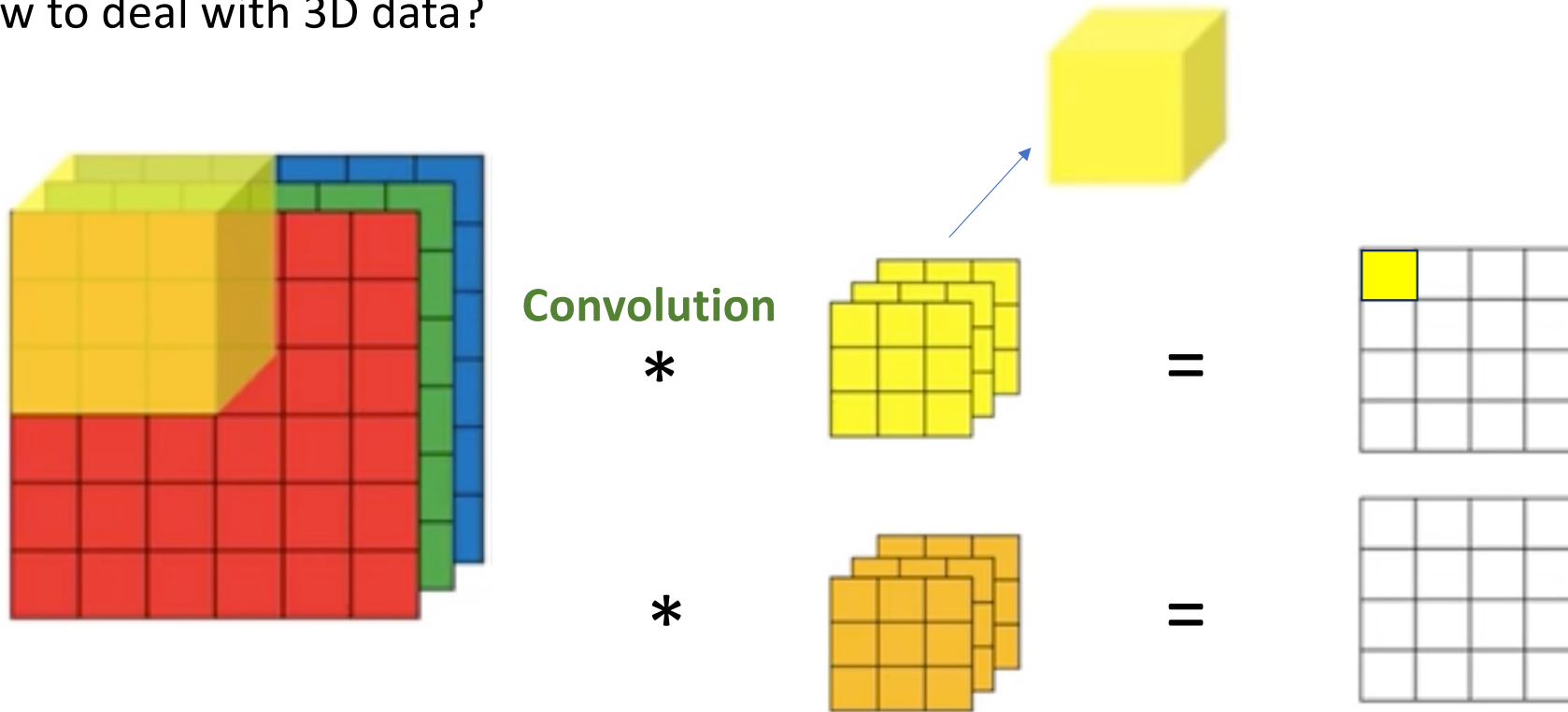
$$r = \left\lfloor \frac{n + 2p - f}{s} + 1 \right\rfloor$$

Example: $n = 7$; $f = 3$; $p = 0$; $s = 3$ $\Rightarrow$ $r = 2$

Image Processing & Computer Vision

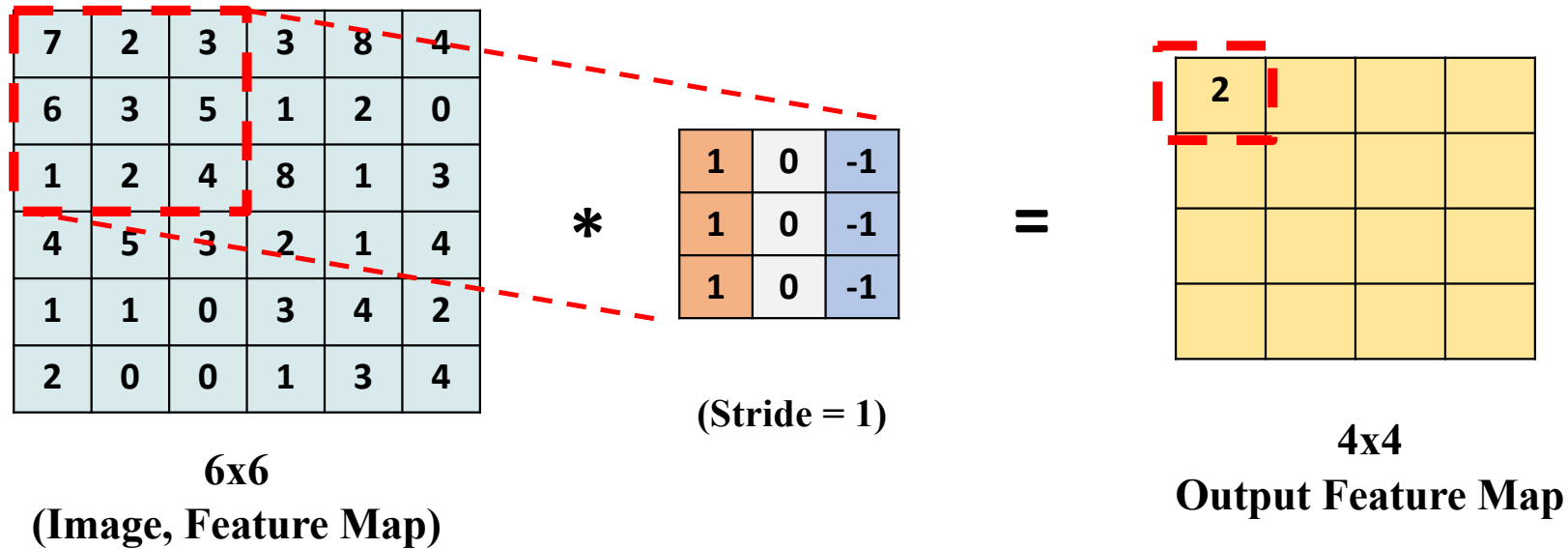
Convolutions over volumes

Convolution seen so far is a 2D convultion.
How to deal with 3D data?



The convolution

2D Convolution applied on a gray scale image

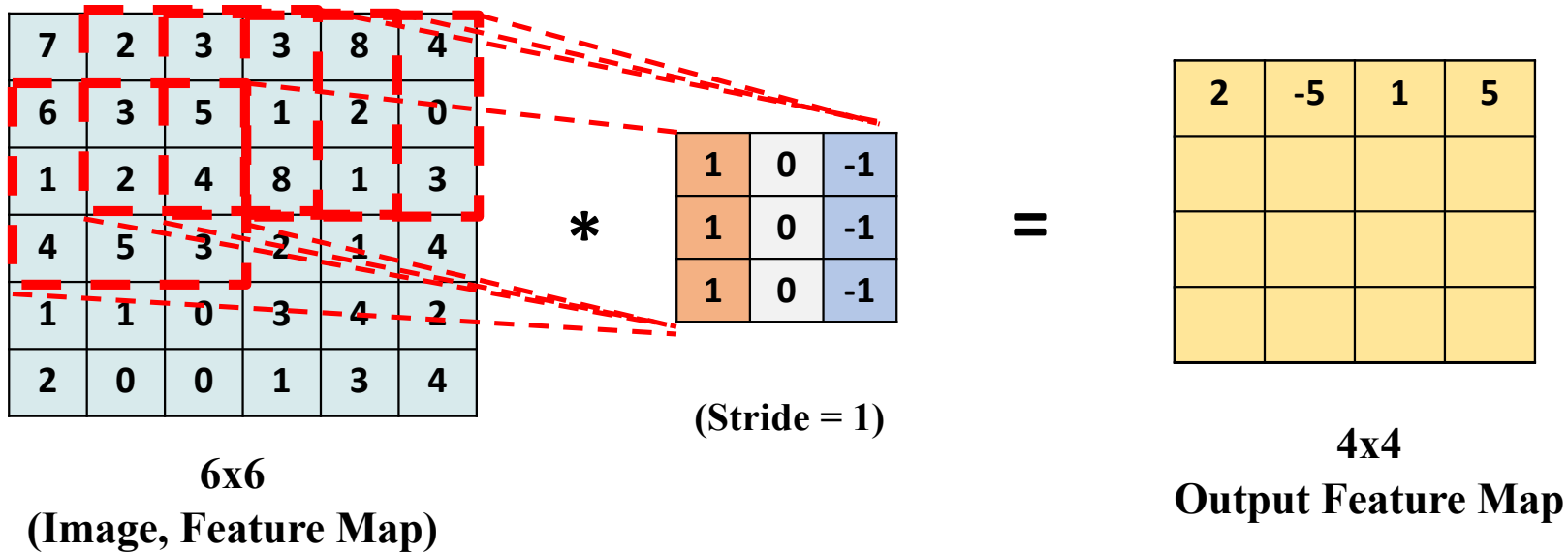


Computing the product between the filter and the image:

$$\begin{aligned} &1 * 7 + 0 * 2 - 1 * 3 + \\ &1 * 6 + 0 * 3 - 1 * 5 + \\ &1 * 1 + 0 * 2 - 1 * 4 = 2 \end{aligned}$$

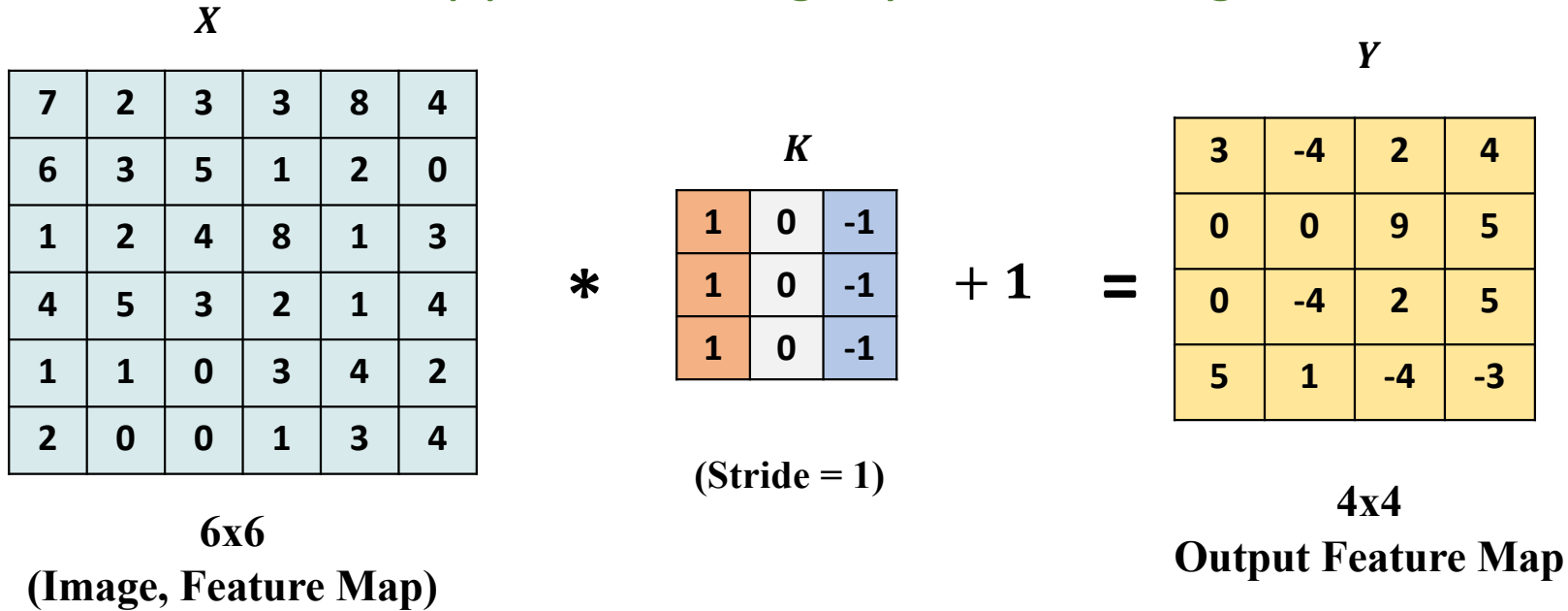
The convolution

2D Convolution applied on a gray scale image



The convolution

2D Convolution applied on a gray scale image



$$Y_{i,j} = \left(\sum_{a=0}^{k-1} \sum_{b=0}^{k-1} X_{i+a,j+b} \times K_{a,b} \right) + bias$$

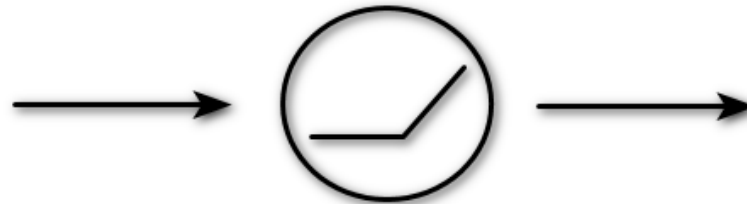
(**kxk**: the size of the filter)

Activation function: ReLU

**Feature Map
before activation**

3	-4	2	4
0	0	9	5
0	-4	2	5
5	1	-4	-3

ReLU



**Feature Map
after activation**

3	0	2	4
0	0	9	5
0	0	2	5
5	1	0	0

$$Y_{i,j} = \sigma\left(\sum_{a=0}^{k-1} \sum_{b=0}^{k-1} X_{i+a,j+b} \times K_{a,b}\right) + bias$$

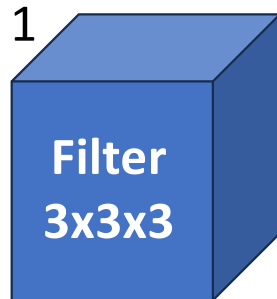
(**k**×**k**: Filter size)

(**σ**: Activation function)

A one layer of convolution

The number of parameters

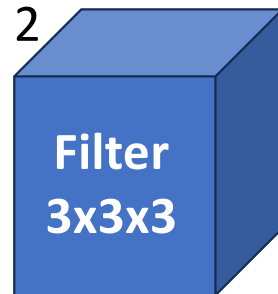
- If you have 10 filters that are 3 x 3 x 3 in one convolutional layer, what is the number of parameters of that layer?



27 parameters
(for convolution)

1 bias

Total = 28 param.

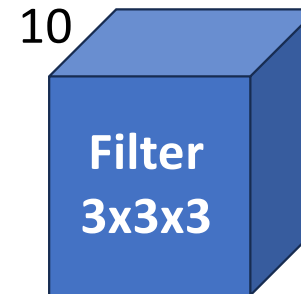


27 parameters
(for convolution)

1 bias

Total = 28 param.

...



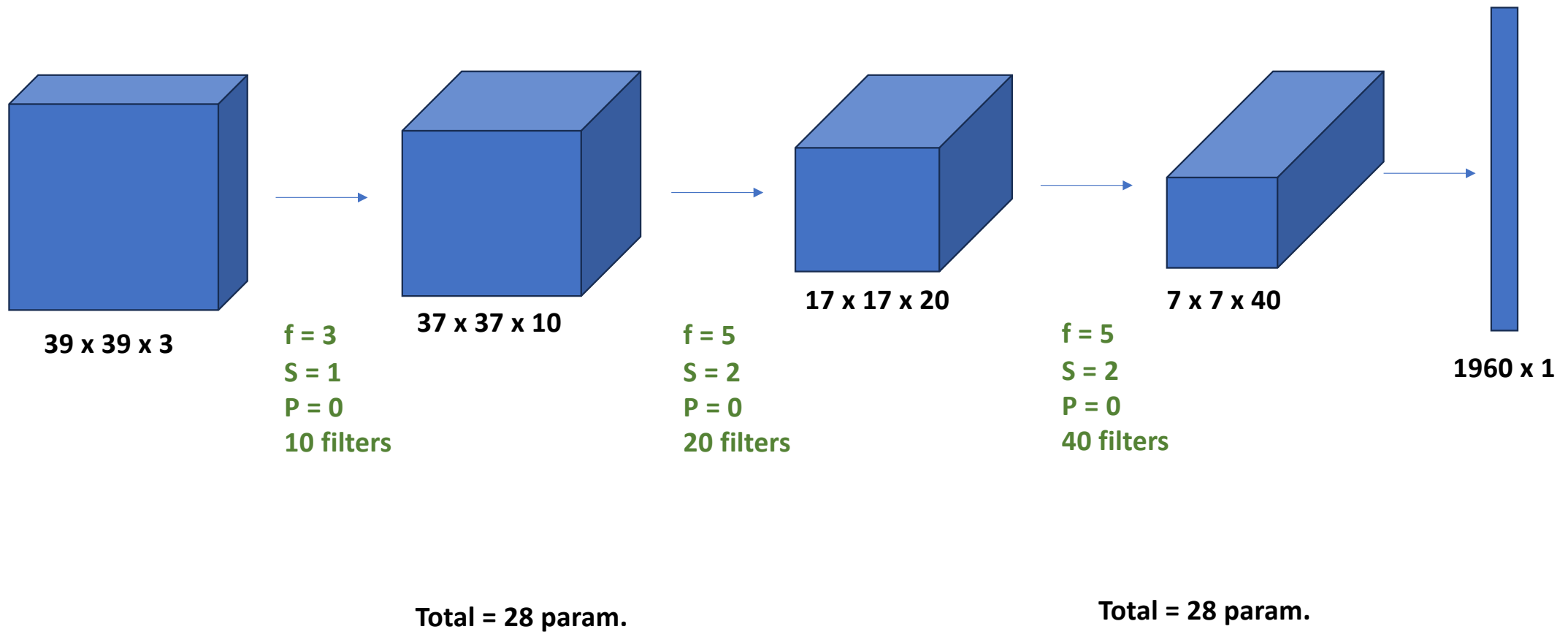
27 parameters
(for convolution)

1 bias

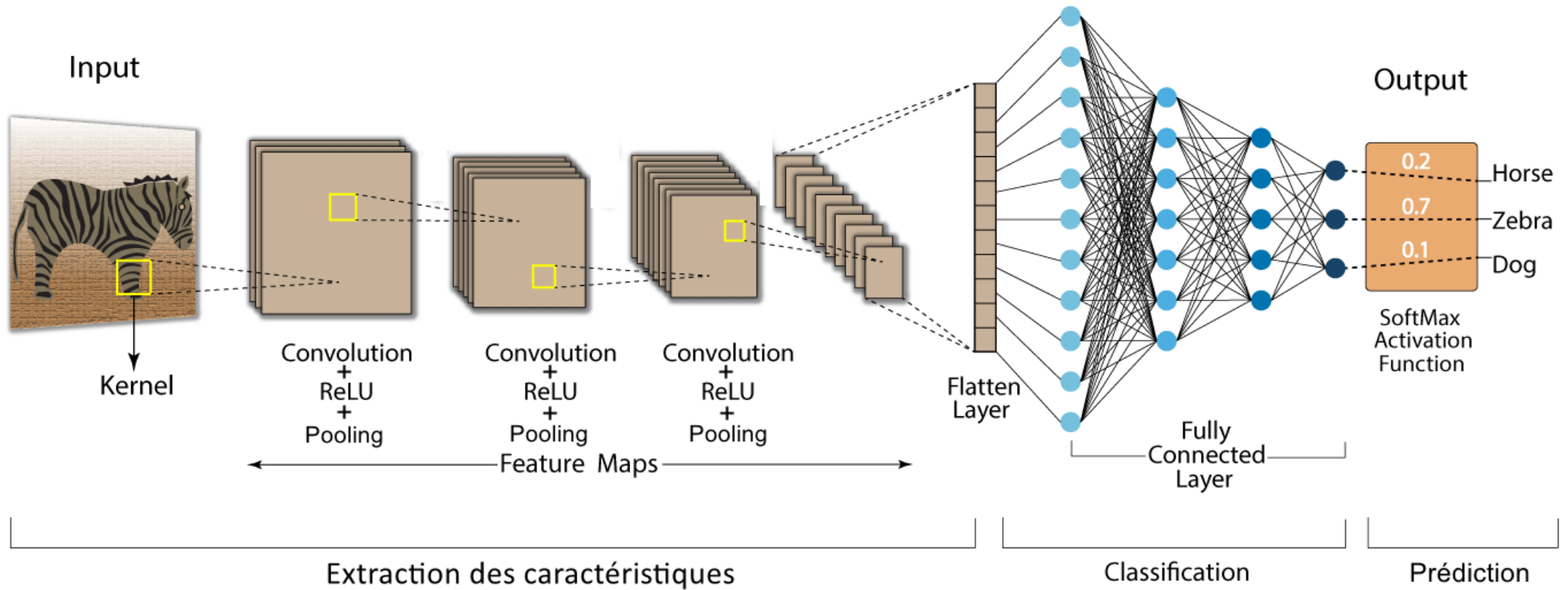
Total = 28 param.

280 parameters

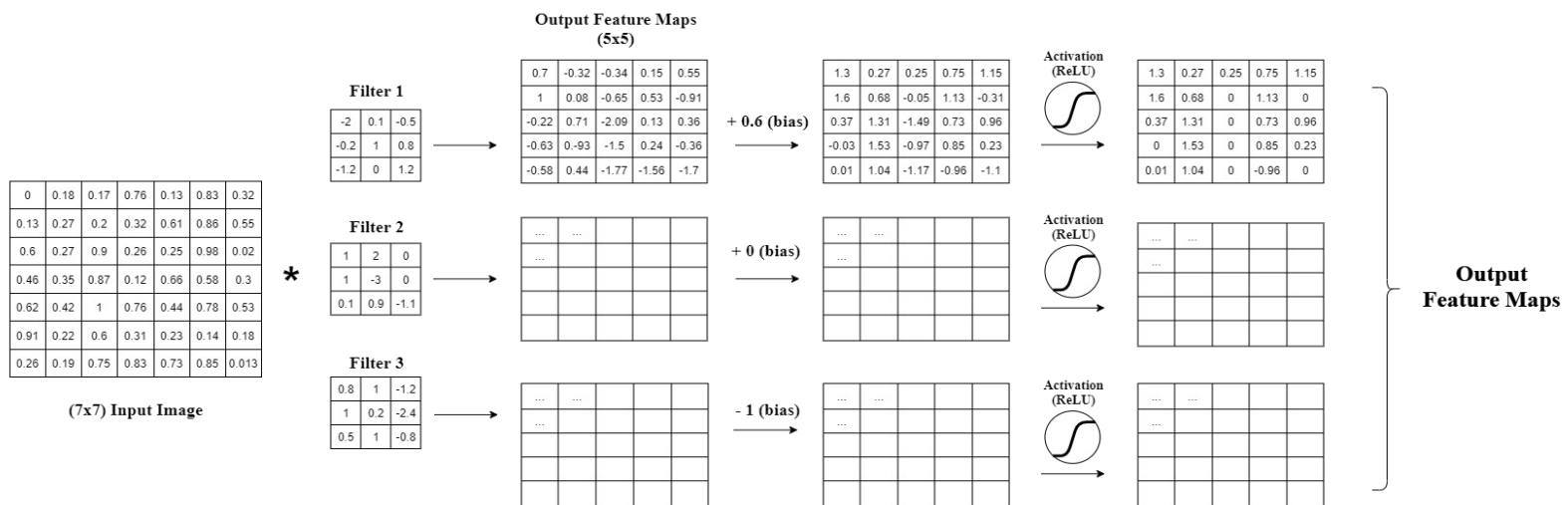
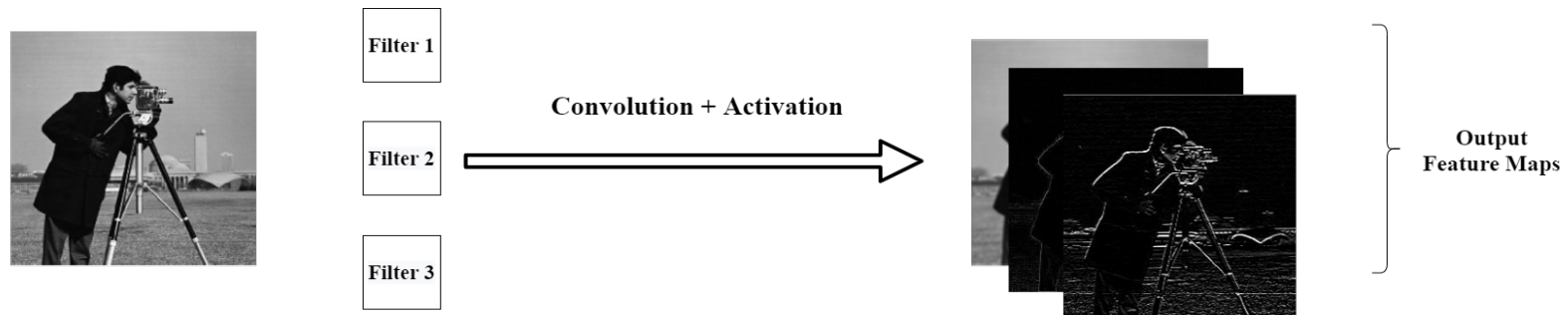
A simple CNN



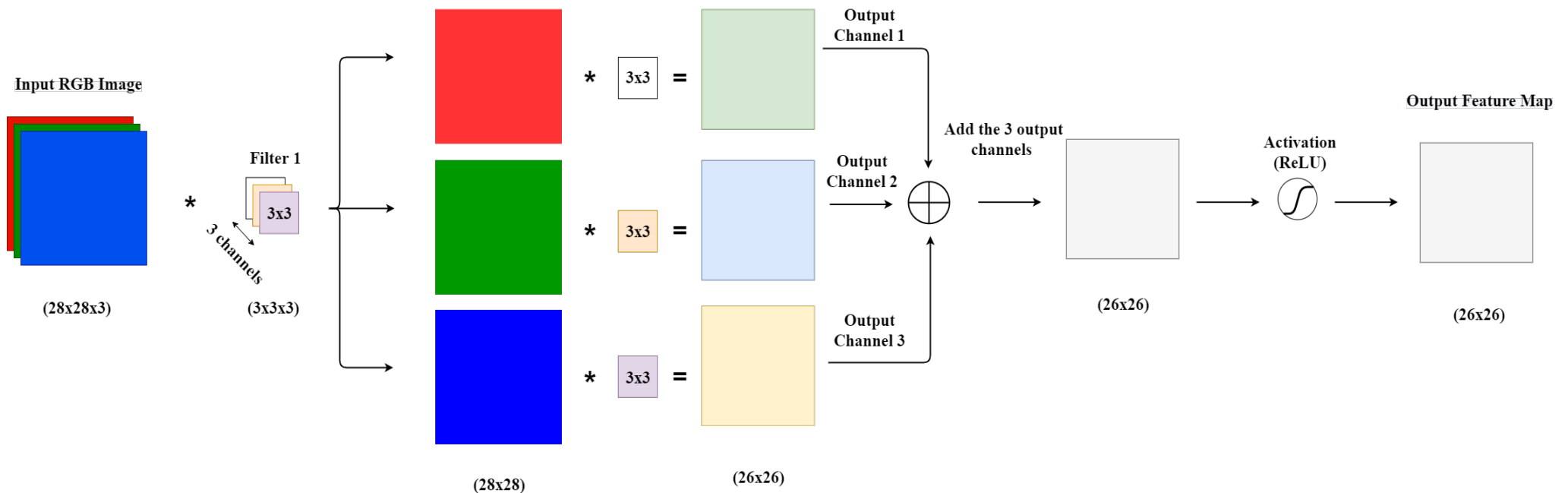
Architecture of a CNN



Architecture of a CNN



Convolution: Multi Channel Input (RGB Image, Multi-feature map channels)



$$Y_{i,j} = \sigma \left(\sum_{c=0}^{C-1} \sum_{a=0}^{k-1} \sum_{b=0}^{k-1} X_{c,i+a,j+b} \times K_{c,a,b} \right) + bias$$

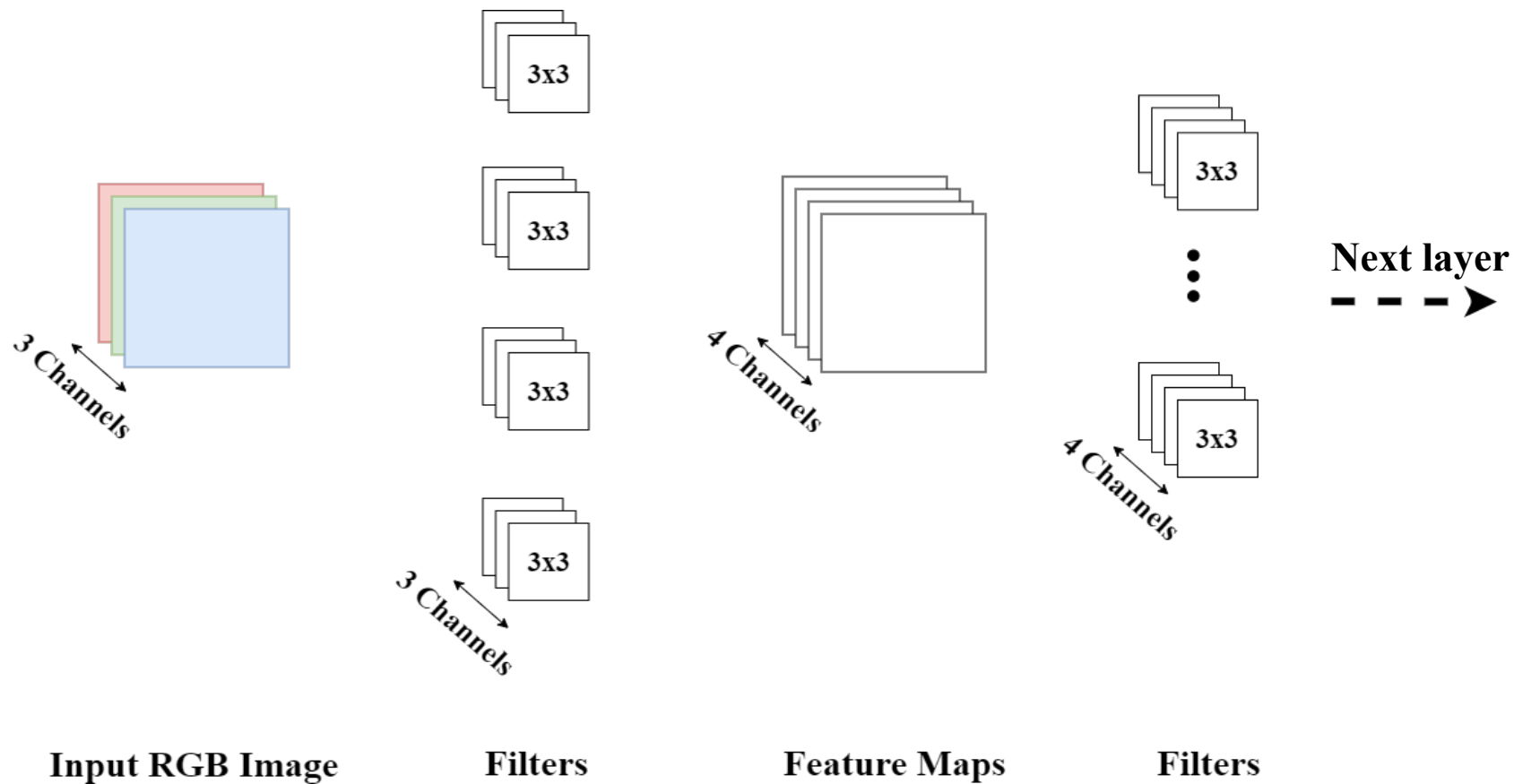
C : Number of channels

$k \times k$: Filter size.

σ : Activation function.

Convolution: Multi Channel Input

(RGB Image, Multi-feature map channels)



Abstraction: Pooling layer

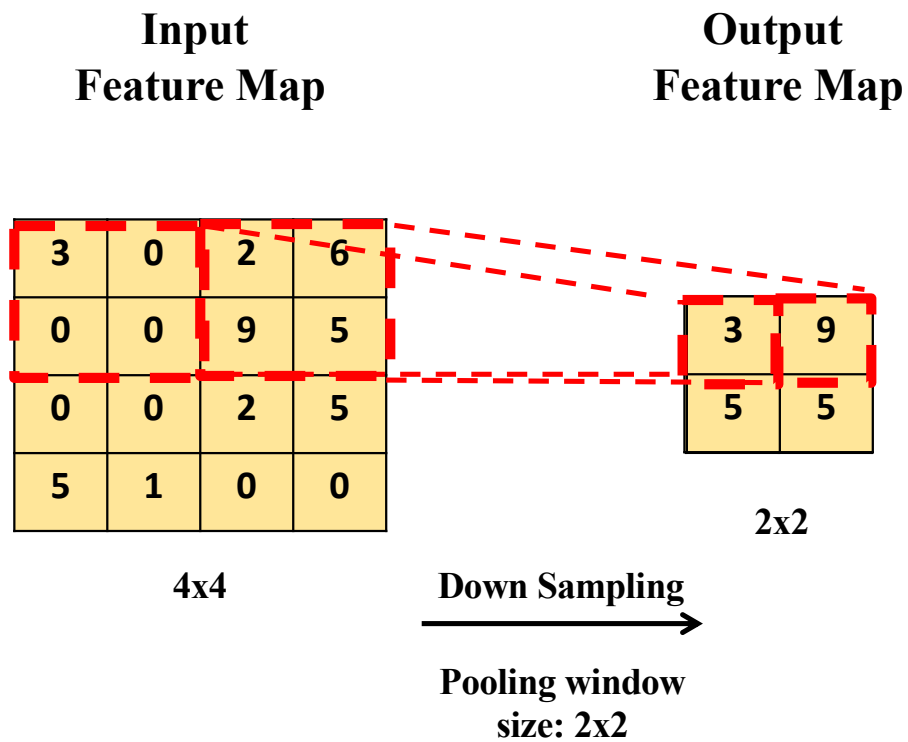
Principle

- Pooling aims to perform data abstraction -- Also called « *Down Sampling* ».
- Pooling allows to reduce the quantity of data while keeping important features => Keeping only useful information.
- Idea: make sur that useful features reach the deep layers, while filtering the rest of the features.
- The more important features reach deeper layers, the better the classification becomes.
- A pooling layer is usually inserted between each two consecutive convolution layers.

Different types of Pooling

1. **Max Pooling.**
2. **Average Pooling.**

Abstraction: MaxPooling layer



Next convolution layer



Note: The size of the pooling window (2x2) has nothing to do with the size of the output feature map (here 2x2)

Abstraction: MaxPooling layer

**Input
Feature Map**

2	0	1	5	2	2
0	0	8	4	1	0
0	0	1	4	8	0
4	0	0	0	1	5
2	1	0	1	3	6
1	4	3	2	9	8

6x6

**Output
Feature Map**

2	8	2
4	4	8
4	3	9

3x3

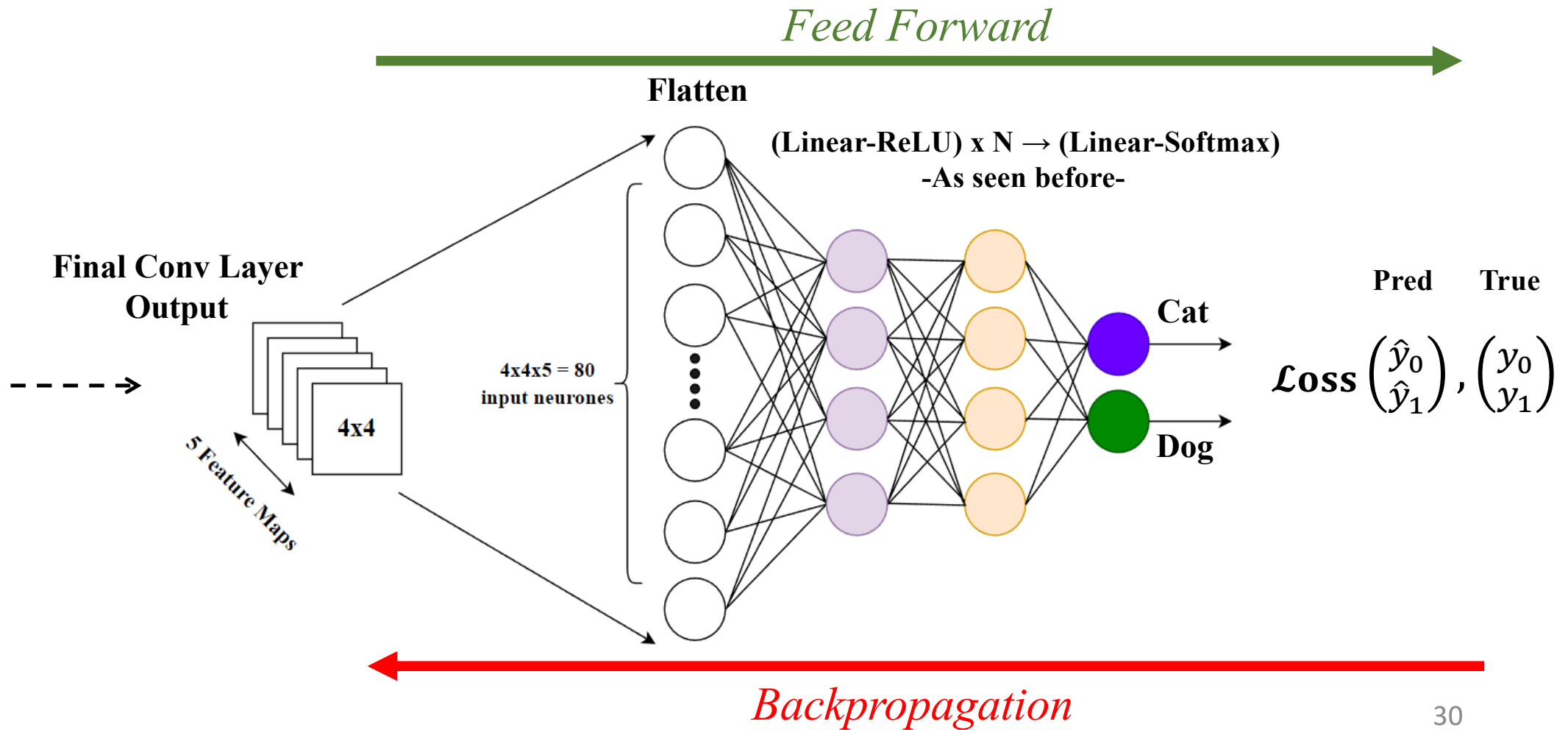
Down Sampling
→

Pooling window
size: 2x2

Next convolution layer



Classification: Fully connected layers



The concept of « Weight Sharing »

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

Input X

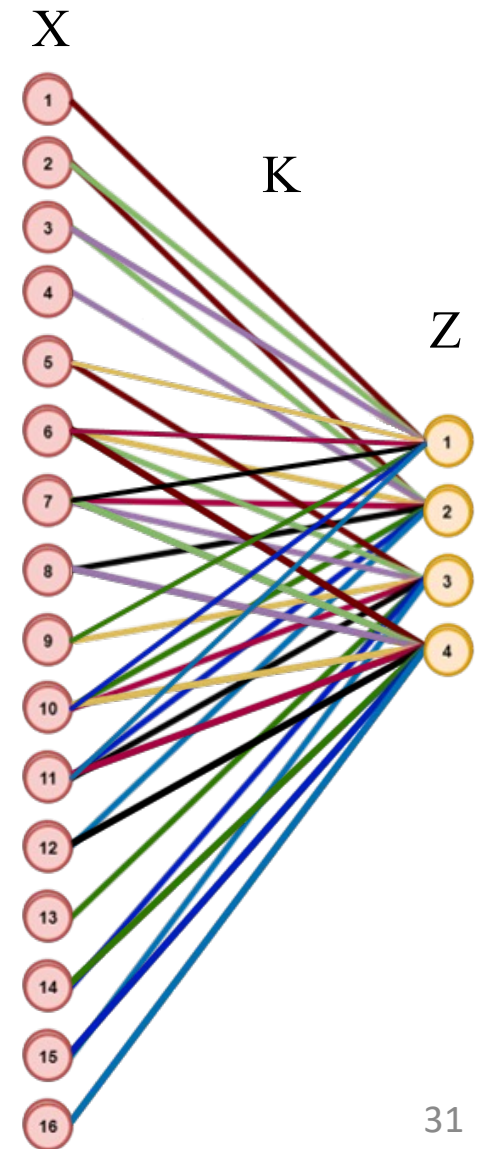


K

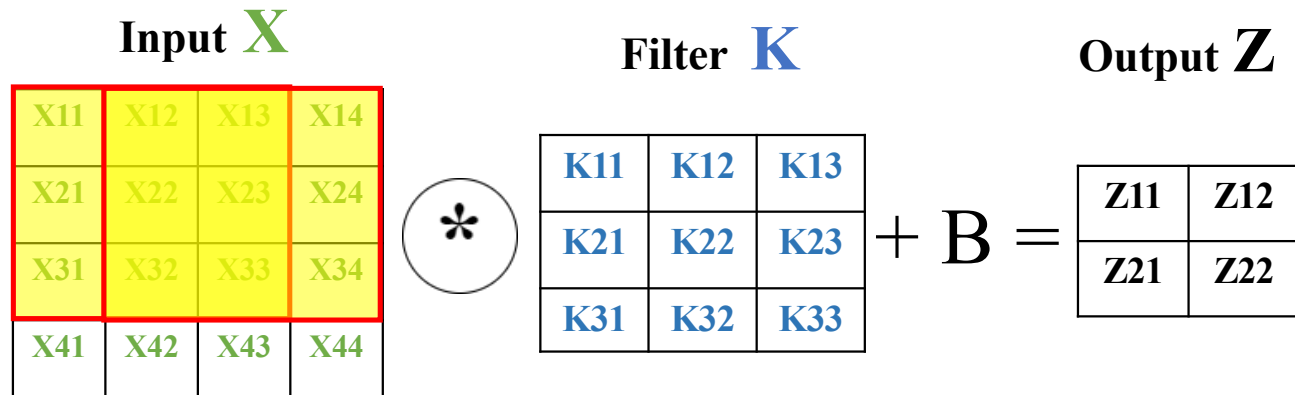
=

1	2
3	4

Output Z



Back-progapagation

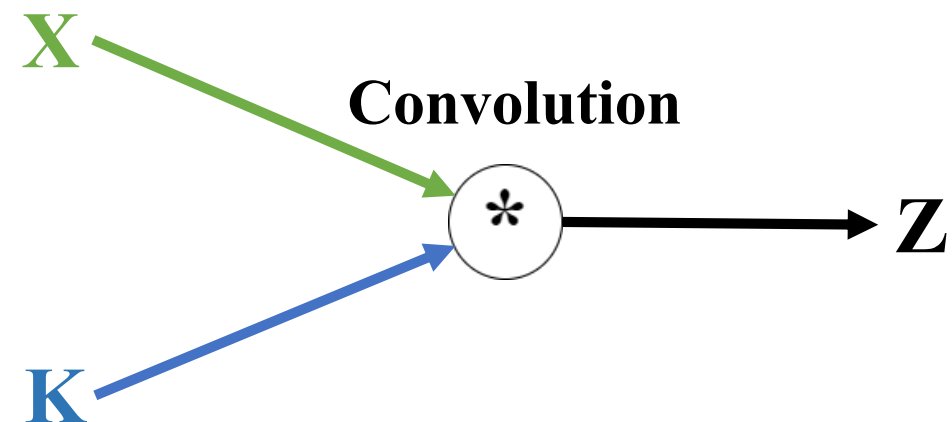


$$Z_{11} = K_{11} * X_{11} + K_{12} * X_{12} + \dots + B$$

$$Z_{12} = K_{11} * X_{12} + K_{12} * X_{13} + \dots + B$$

$$Z_{21} = K_{11} * X_{21} + K_{12} * X_{22} + \dots + B$$

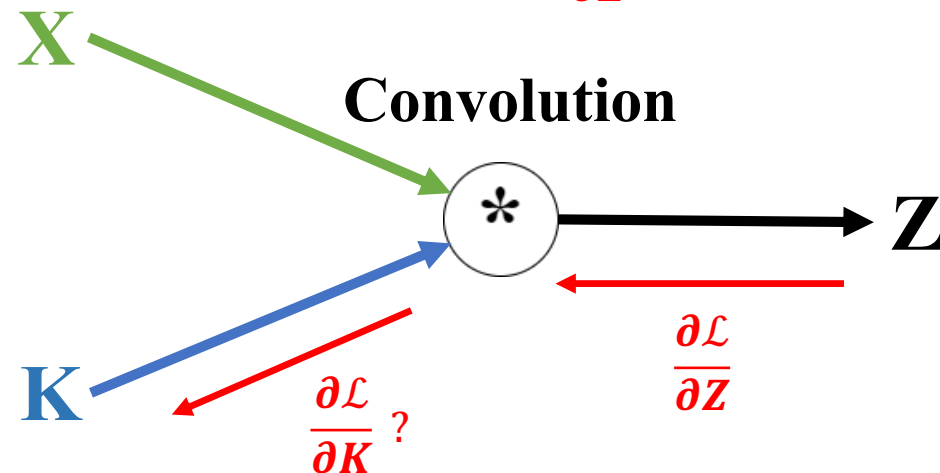
$$Z_{22} = K_{11} * X_{22} + K_{12} * X_{23} + \dots + B$$



Back-propagation

Note: We already possess the gradient of the loss computed from the

next layer : $\frac{\partial \mathcal{L}}{\partial Z}$



$$Z_{11} = K_{11} * X_{11} + K_{12} * X_{12} + \dots + B$$

$$Z_{12} = K_{11} * X_{12} + K_{12} * X_{13} + \dots + B$$

$$Z_{21} = K_{11} * X_{21} + K_{12} * X_{22} + \dots + B$$

$$Z_{22} = K_{11} * X_{22} + K_{12} * X_{23} + \dots + B$$

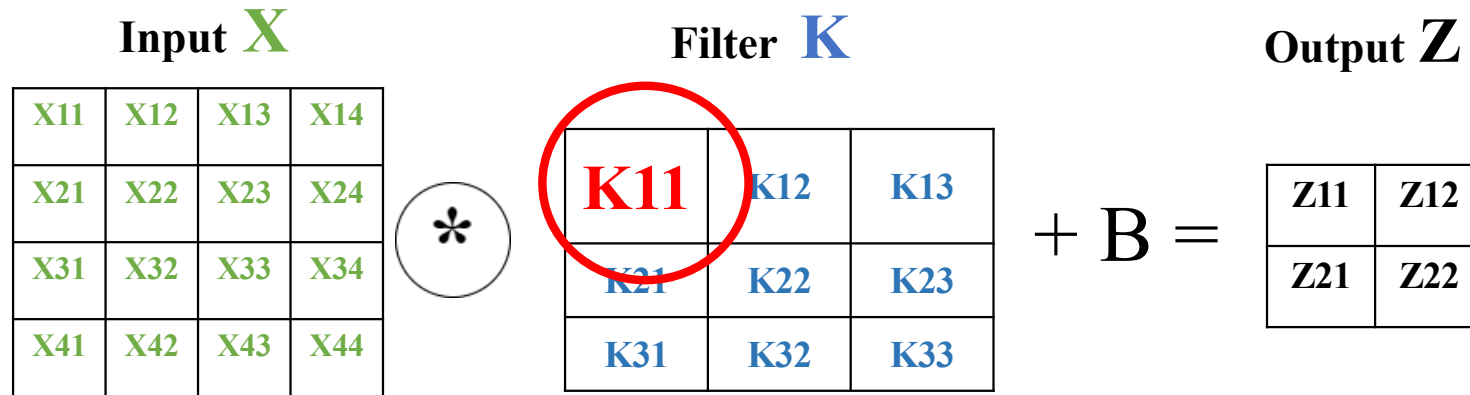
How to adjust the filter weights in order to minimize the output loss?

This involves calculating the partial derivative of the

loss with respect to the specific kernel $\frac{\partial \mathcal{L}}{\partial K}$

Back-progapagation

Example of tuning one parameter K_{11} (The same method for all K_{ij})



$$Z_{11} = K_{11} * X_{11} + K_{12} * X_{12} + \dots + B$$

$$Z_{12} = K_{11} * X_{12} + K_{12} * X_{13} + \dots + B$$

$$Z_{21} = K_{11} * X_{21} + K_{12} * X_{22} + \dots + B$$

$$Z_{22} = K_{11} * X_{22} + K_{12} * X_{23} + \dots + B$$

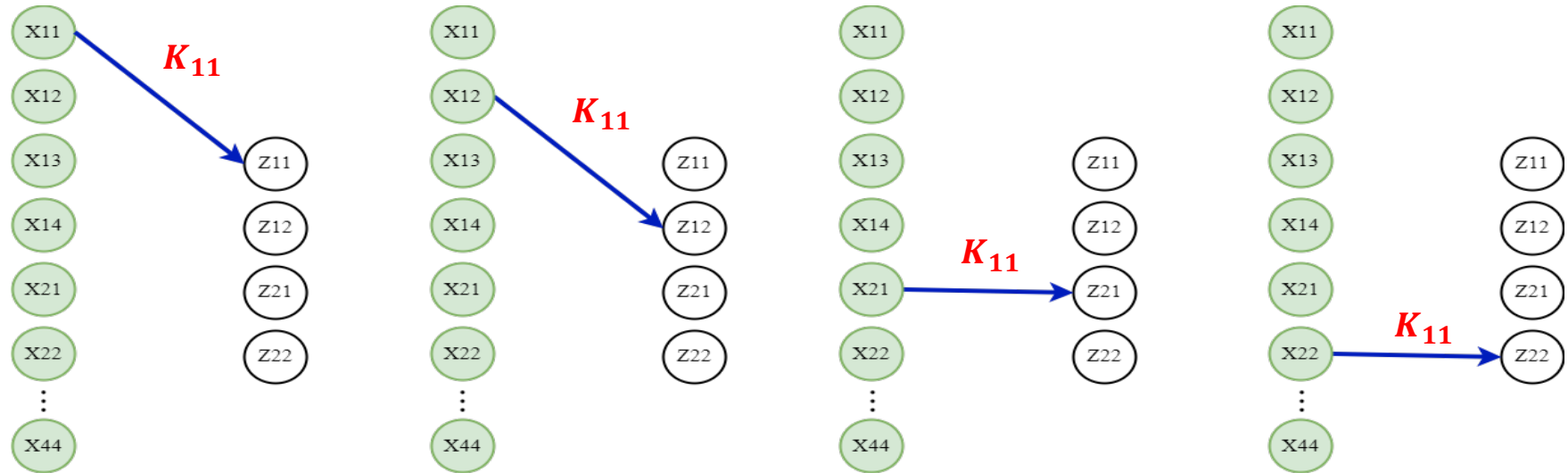
How to adjust the filter weights in order to minimize the output loss?

This involves calculating the partial derivative of the

loss with respect to the specific kernel $\frac{\partial \mathcal{L}}{\partial K}$

Back-progapagation

Example of tuning one parameter K_{11} (The same method for all K_{ij})



$$\begin{aligned} Z_{11} &= K_{11} * X_{11} + K_{12} * X_{12} + \dots + B \\ Z_{12} &= K_{11} * X_{12} + K_{12} * X_{13} + \dots + B \\ Z_{21} &= K_{11} * X_{21} + K_{12} * X_{22} + \dots + B \\ Z_{22} &= K_{11} * X_{22} + K_{12} * X_{23} + \dots + B \end{aligned}$$

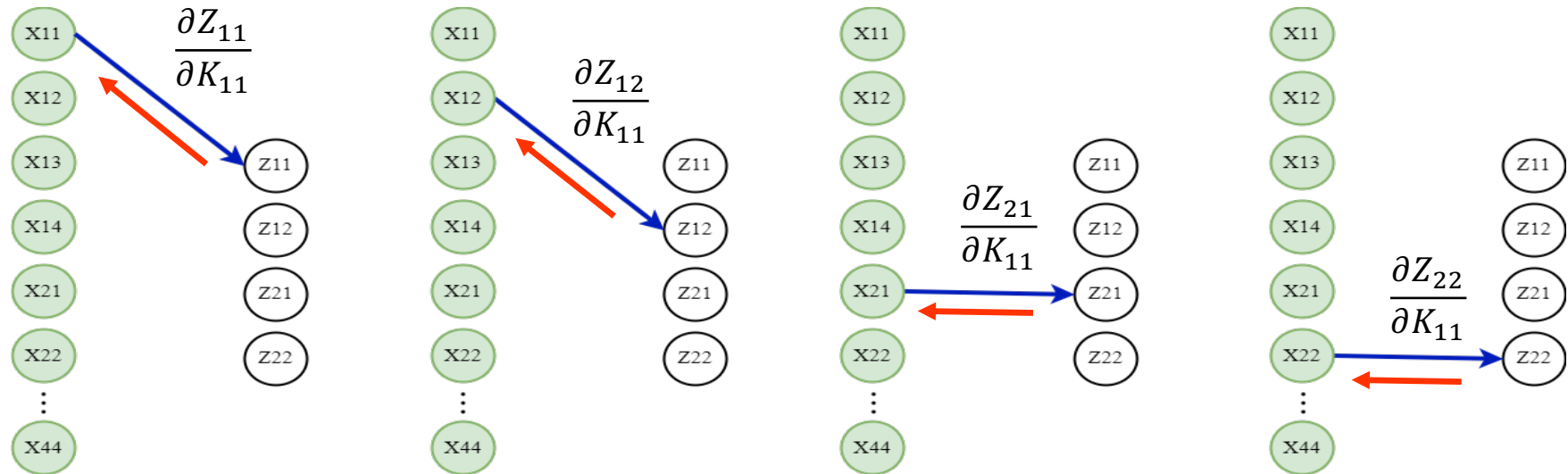
The partial loss has been previously calculated

Need to be computed from output equations Z_{ij}

$$\frac{\partial \mathcal{L}}{\partial K} = \frac{\partial \mathcal{L}}{\partial Z} * \frac{\partial Z}{\partial K}$$

Back-progapagation

Example of tuning one parameter K_{11} (The same method for all K_{ij})



$$Z_{11} = K_{11} * X_{11} + K_{12} * X_{12} + \dots + B$$

$$Z_{12} = K_{11} * X_{12} + K_{12} * X_{13} + \dots + B$$

$$Z_{21} = K_{11} * X_{21} + K_{12} * X_{22} + \dots + B$$

$$Z_{22} = K_{11} * X_{22} + K_{12} * X_{23} + \dots + B$$

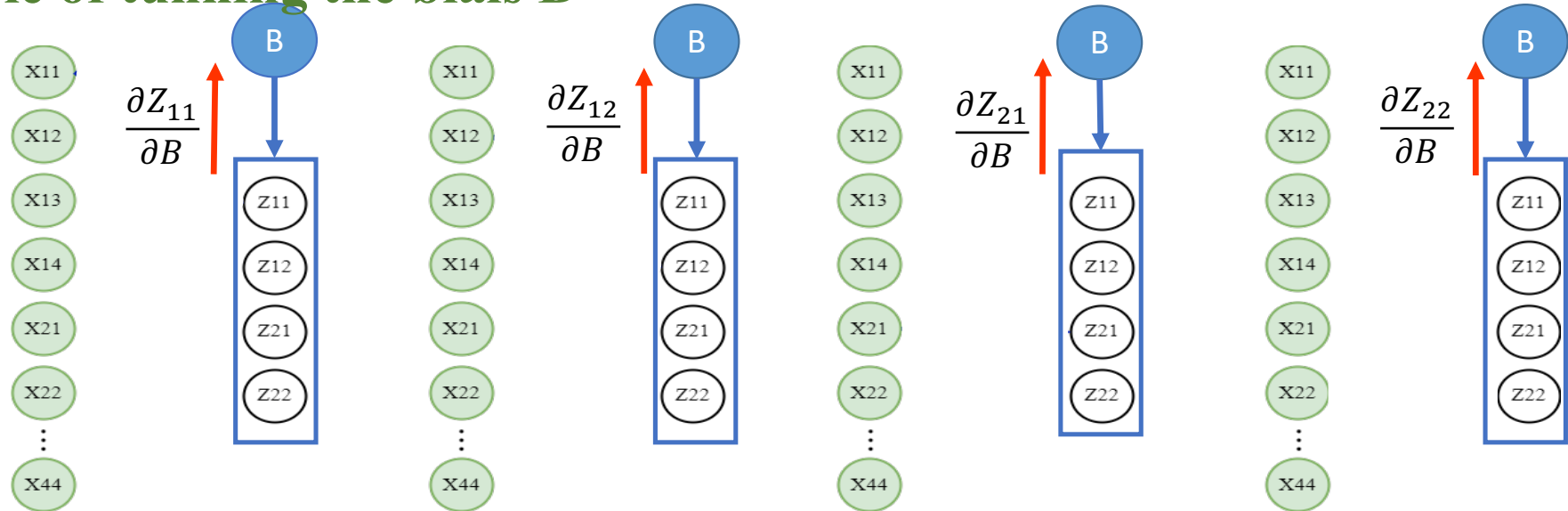
$$\frac{\partial \mathcal{L}}{\partial K_{11}} = \underbrace{\frac{\partial \mathcal{L}}{\partial Z_{11}} * \frac{\partial Z_{11}}{\partial K_{11}}}_{\text{From } Z_{11}} + \underbrace{\frac{\partial \mathcal{L}}{\partial Z_{12}} * \frac{\partial Z_{12}}{\partial K_{11}}}_{\text{From } Z_{12}} + \underbrace{\frac{\partial \mathcal{L}}{\partial Z_{21}} * \frac{\partial Z_{21}}{\partial K_{11}}}_{\text{From } Z_{21}} + \underbrace{\frac{\partial \mathcal{L}}{\partial Z_{22}} * \frac{\partial Z_{22}}{\partial K_{11}}}_{\text{From } Z_{22}}$$

$$\quad \quad \quad \parallel \quad \quad \quad \parallel \quad \quad \quad \parallel \quad \quad \quad \parallel$$

$$\quad \quad \quad X_{11} \quad \quad \quad X_{12} \quad \quad \quad X_{21} \quad \quad \quad X_{22} \quad 36$$

Back-progapagation

Example of tuning the biaiis B



$$Z_{11} = K_{11} * X_{11} + K_{12} * X_{12} + \dots + \mathbf{B}$$

$$Z_{12} = K_{11} * X_{12} + K_{12} * X_{13} + \dots + \mathbf{B}$$

$$Z_{21} = K_{11} * X_{21} + K_{12} * X_{22} + \dots + \mathbf{B}$$

$$Z_{22} = K_{11} * X_{22} + K_{12} * X_{23} + \dots + \mathbf{B}$$

$$\frac{\partial \mathcal{L}}{\partial B} = \underbrace{\frac{\partial \mathcal{L}}{\partial Z_{11}} * \frac{\partial Z_{11}}{\partial B}}_{\substack{\text{From } Z_{11} \\ \parallel \\ 1}} + \underbrace{\frac{\partial \mathcal{L}}{\partial Z_{12}} * \frac{\partial Z_{12}}{\partial B}}_{\substack{\text{From } Z_{12} \\ \parallel \\ 1}} + \underbrace{\frac{\partial \mathcal{L}}{\partial Z_{21}} * \frac{\partial Z_{21}}{\partial B}}_{\substack{\text{From } Z_{21} \\ \parallel \\ 1}} + \underbrace{\frac{\partial \mathcal{L}}{\partial Z_{22}} * \frac{\partial Z_{22}}{\partial B}}_{\substack{\text{From } Z_{22} \\ \parallel \\ 1}}$$

Back-progapagation

Updating the K_{11} weight, as well as the bias B

$$K_{11} = K_{11} - \frac{\partial \mathcal{L}}{\partial K_{11}} * learning_rate$$

$$B = B - \frac{\partial \mathcal{L}}{\partial B} * learning_rate$$

General expression for $N \times N$ output

$$\frac{\partial \mathcal{L}}{\partial K_{a,b}} = \sum_{i=1}^N \sum_{j=1}^N \frac{\partial \mathcal{L}}{\partial Z_{i,j}} * \frac{\partial Z_{i,j}}{\partial K_{a,b}} \qquad \frac{\partial \mathcal{L}}{\partial B} = \left(\sum_{i=1}^N \sum_{j=1}^N \frac{\partial \mathcal{L}}{\partial Z_{i,j}} * 1 \right)$$

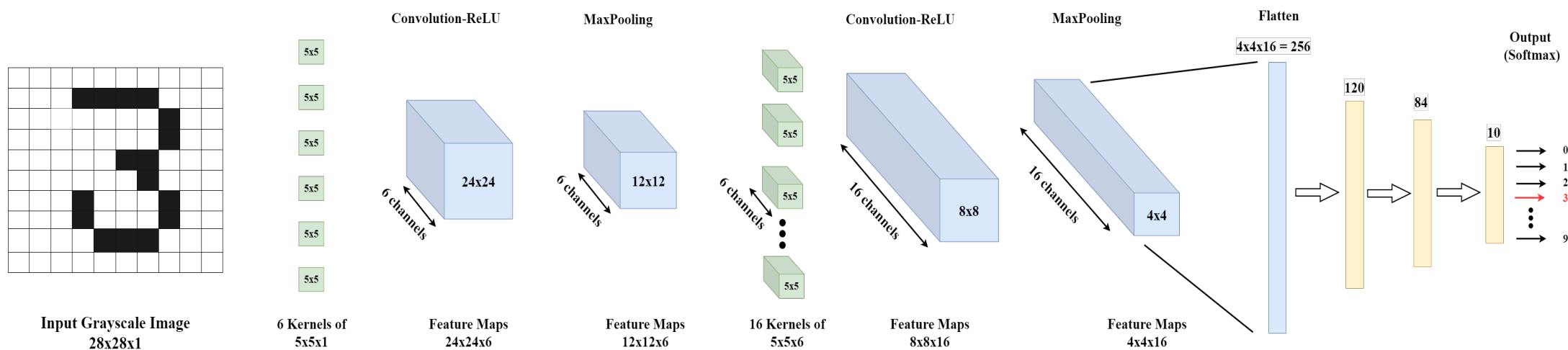
far1din

Architecture of the LeNet-5 CNN model

LeNet-5 is a CNN model that has been proposed by Yann Le Cun in 1998.
Dedicated to handwritten character recognition.

Layer	# Filters/neurones	Filter size	Bias	Feature Map size	Activation
Input	-	-	-	28x28x1	-
Conv1	6	5x5	6	24x24x6	ReLU
MaxPool2D	-	2x2	-	12x12x6	-
Conv2	16	5x5	16	8x8x16	ReLU
MaxPool2D	-	2x2	-	4x4x16	-
Flatten	256	-	256	256	-
Dense	120x256	-	120	120	ReLU
Dense	84x120	-	84	84	ReLU
Dense	10x84	-	10	10	Softmax

Architecture of the LeNet-5 CNN model



A visual example on CIFAR-10 dataset

<https://cs.stanford.edu/people/karpathy/convnetjs/demo/cifar10.html>

